

Documentation Technique — Projet SANGEL (Odoo 19)

Version : 19.0

Auteur : Partenaires Succes / Adama KONE

Date : Mai 2026

Licence : LGPL-3 / AGPL-3

Table des matières

1. [Vue d'ensemble du projet](#)
2. [Architecture générale](#)
3. [Inventaire des modules](#)
4. [Détail des modules](#)
 - [4.1 custom_pos — Point de Vente personnalisé](#)
 - [4.2 custom_sales — Ventes et promotions](#)
 - [4.3 custom_stock — Gestion des stocks](#)
 - [4.4 custom_loyalty — Programme de fidélité](#)
 - [4.5 custom_food_credit — Crédit alimentaire](#)
 - [4.6 custom_api_sage_x3 — Intégration SAGE X3](#)
 - [4.7 custom_purchase — Achats](#)
 - [4.8 custom_account — Comptabilité](#)
 - [4.9 custom_partner — Partenaires](#)
 - [4.10 custom_reports — Rapports PDF](#)
 - [4.11 custom_reliquat_report — Rapport de reliquats](#)
 - [4.12 custom_multi_barcode_for_products — Multi-codes-barres](#)
 - [4.13 custom_price_change_tracker — Suivi des prix](#)
 - [4.14 cfao_dashboard_report — Tableau de bord CFAO](#)
 - [4.15 dashboard_management_administration — Dashboard Admin](#)
 - [4.16 fne_certification — Certification FNE](#)
 - [4.17 stock_avco_correction — Correction AVCO](#)
 - [4.18 pos_history_import — Import historique POS](#)
 - [4.19 hide_menu_user — Masquage de menus](#)
 - [4.20 queue_job — File de travaux asynchrones](#)

- 5. [Modèles de données clés](#)
 - 6. [Workflows métier](#)
 - 7. [Sécurité et droits d'accès](#)
 - 8. [APIs et intégrations externes](#)
 - 9. [Personnalisations du POS \(JavaScript\)](#)
 - 10. [Rapports et tableaux de bord](#)
 - 11. [Tâches planifiées \(Crons\)](#)
 - 12. [Dépendances externes Python](#)
 - 13. [Paramètres système](#)
-

1. Vue d'ensemble du projet

Le projet SANGEL est une suite de **24 modules Odoo 19 personnalisés** destinée à la gestion complète d'une enseigne de distribution multi-sites. Elle couvre :

- La **caisse (POS)** avec authentification sécurisée, taxes automatiques, multi-devises et fidélité
 - La **gestion des stocks** avec inventaires physiques, transferts inter-sociétés et pilotage de coûts AVCO
 - Les **ventes et promotions** avec règles tarifaires dynamiques et gestion de remises
 - Les **achats** avec synchronisation vers l'ERP SAGE X3
 - La **fidélité clients** avec familles de points configurables
 - Le **crédit alimentaire** pour les employés
 - La **certification FNE** des factures
 - Des **tableaux de bord et rapports PDF** élaborés
-

2. Architecture générale

```
addons-sangel-19/
├─ custom_pos/                # POS – cœur caisse
├─ custom_sales/              # Ventes, promotions, tarifs
├─ custom_stock/              # Stocks, inventaires, inter-sociétés
├─ custom_loyalty/            # Fidélité clients
├─ custom_food_credit/         # Crédit alimentaire employés
├─ custom_api_sage_x3/         # Intégration ERP SAGE X3
├─ custom_purchase/           # Achats personnalisés
├─ custom_account/            # Comptabilité
├─ custom_partner/            # Partenaires/clients
├─ custom_reports/            # Rapports PDF (20+)
├─ custom_reliquat_report/     # Rapports de reliquats
├─ custom_multi_barcode_for_products/ # Multi-codes-barres
├─ custom_price_change_tracker/ # Suivi historique des prix
├─ cfao_dashboard_report/      # Tableau de bord multi-société
├─ dashboard_management_administration/ # Dashboard admin
├─ fne_certification/          # Certification fiscale FNE
├─ stock_avco_correction/      # Correction coûts AVCO
├─ pos_history_import/         # Import historique POS
├─ hide_menu_user/            # Masquage de menus par utilisateur
└─ queue_job/                 # File de tâches asynchrones (OCA)
```

Dépendances inter-modules (principales) :

```
custom_pos ← custom_loyalty ← custom_food_credit
custom_stock ← custom_sales ← custom_purchase
custom_api_sage_x3 ← custom_stock, custom_partner, custom_food_credit
dashboard_management_administration ← custom_api_sage_x3, custom_stock, custom_pos
custom_reports ← custom_stock, custom_sales, dashboard_management_administration
```

3. Inventaire des modules

Module	Version	Rôle principal
<code>custom_pos</code>	1.0.0	POS : authentification, taxes, multi-devises, clôture
<code>custom_sales</code>	1.0.0	Promotions, tarifs, remises globales POS
<code>custom_stock</code>	latest	Inventaires physiques, transferts, AVCO
<code>custom_loyalty</code>	1.2.0	Familles de fidélité, points POS
<code>custom_food_credit</code>	1.1.0	Crédit alimentaire employés
<code>custom_api_sage_x3</code>	1.0	Synchronisation avec SAGE X3
<code>custom_purchase</code>	1.0.0	Achats, réapprovisionnement
<code>custom_account</code>	latest	Budgets analytiques, comptabilité
<code>custom_partner</code>	1.1.0	Extensions partenaires
<code>custom_reports</code>	latest	Rapports PDF (20+)
<code>custom_reliquat_report</code>	latest	Taux de satisfaction commandes
<code>custom_multi_barcode_for_products</code>	latest	Multi-codes-barres produits
<code>custom_price_change_tracker</code>	latest	Historique et alertes de prix
<code>cfao_dashboard_report</code>	19.0.1.0.0	Dashboard ventes/marges multi-société
<code>dashboard_management_administration</code>	latest	Dashboard admin avec graphiques JS
<code>fne_certification</code>	1.0	Certification fiscale FNE
<code>stock_avco_correction</code>	19.0.1.0.0	Correction AVCO post-migration
<code>pos_history_import</code>	19.0.1.0.0	Import Excel historique POS
<code>hide_menu_user</code>	19.0.1.0.0	Masquage de menus par utilisateur
<code>queue_job</code>	19.0.1.1.0	File d'attente de tâches (OCA)

4. Détail des modules

4.1 `custom_pos` — Point de Vente personnalisé

Objectif : Personnalisation complète de la caisse Odoo 19 pour les besoins métier SANGEL.

Modèles :

Modèle	Description
<code>pos.manager.code</code>	Codes d'authentification manager pour les opérations sensibles
<code>pos.order</code> (hérit.)	Override de <code>write()</code> pour autoriser la modification du paiement sur commandes imprimées
<code>pos.config</code> (hérit.)	Configuration étendue de la caisse
<code>pos.session</code> (hérit.)	Rapports de clôture (cloture_caisse, rapport_validation)
<code>pos.payment_method</code> (hérit.)	Méthodes de paiement étendues
<code>product.template</code> (hérit.)	Paramètres POS produit
<code>account.tax</code> (hérit.)	Application automatique de la taxe AIRSI

Contrôleurs JSON :

Route	Description
<code>/pos_promo/check_3x4</code>	Vérification de la promotion 3×4 (3 achetés = 4 livrés)
<code>/pos/active_currencies</code>	Liste des devises actives avec taux de change
<code>/pos/convert_currency</code>	Conversion devise étrangère → devise société

Groupes de sécurité :

Groupe	Rôle
<code>caissiere</code>	Caissière — accès limité à la saisie
<code>superviseur</code>	Superviseur — supervision et corrections
<code>assistant_magasin</code>	Assistant magasin — ventes + stocks
<code>responsable_magasin</code>	Responsable magasin — accès étendu
<code>commercial</code>	Commercial — ventes et comptes

Fonctionnalités notables :

- Raccourci `Alt+C` pour l'ouverture manuelle du tiroir-caisse
- Popup de conversion multi-devises en temps réel
- Application automatique de la taxe AIRSI selon client et produit
- Personnalisation de la navbar (restrictions par rôle caissière/DSI)
- Import de points de fidélité via assistant (wizard)

4.2 `custom_sales` — Ventes et promotions

Objectif : Gestion des promotions, des tarifs et des remises avec calcul dynamique des prix.

Modèles :

Modèle	Champs clés	Description
<code>sale.promotion</code>	<code>name</code> , <code>date_start</code> , <code>date_end</code> , <code>company_ids</code> , <code>line_ids</code> , <code>apply_in_pos</code>	En-tête de promotion
<code>sale.promotion.line</code>	<code>product_id</code> , <code>discount</code> , <code>promo_ht</code> , <code>promo_ttc</code> , <code>promo_pa</code> , <code>coeff</code> , <code>promo_tx_marque</code>	Règle de remise par produit
<code>product.pricelist</code> (hérit.)	—	Extension liste de prix
<code>product.template</code> (hérit.)	—	Prix de vente calculés
<code>sale.order</code> (hérit.)	—	Application automatique des promos
<code>res.partner</code> (hérit.)	—	Lien client → liste de prix

Logique de calcul des prix promotionnels (3 modes) :

```
Mode 1 - Remise %      : saisie discount → calcul auto promo_ht
Mode 2 - Prix HT       : saisie promo_ht → calcul auto discount et promo_ttc
Mode 3 - Prix TTC      : saisie promo_ttc → calcul auto promo_ht et discount
```

Chaque mode utilise des champs inverses (`_inverse_*`) pour assurer la cohérence en temps réel.

Assets POS : `global_discount_patch.js`, `promotion_pos_patch.js`

4.3 `custom_stock` — Gestion des stocks

Objectif : Inventaires physiques, transferts inter-sociétés, gestion AVCO, retours et reporting.

Modèles principaux :

Modèle	Description
<code>physical.inventory</code>	En-tête d'inventaire physique (états : draft → in_progress → done)
<code>physical.inventory.line</code>	Lignes de comptage avec quantité physique
<code>physical.inventory.retour</code>	Suivi des retours d'inventaire
<code>code.inventory</code> / <code>code.category.inventory</code>	Système de codification des inventaires
<code>team.inventory</code>	Équipes d'inventaire
<code>family.inventory</code> / <code>radius.inventory</code>	Familles produit et rayons entrepôt
<code>picking.inter.company</code>	Bons de transfert inter-sociétés
<code>picking.request.company</code>	Demandes de transfert inter-sociétés
<code>stock.picking</code> (hérit.)	Gestion étendue des bons de livraison/réception
<code>stock.scrap.breakers</code>	Suivi des casses et pertes
<code>stock.product.multicompany.view</code>	Vue stock multi-société
<code>stock.avco.report</code>	Rapport AVCO (coût moyen pondéré)
<code>avco.guard</code>	Garde-fou AVCO : limite les écarts à ±10 M FCFA
<code>company.state</code>	Statut des sociétés

Assistants (wizards) :

- `product_select_wizard` — Sélection de produits
- `stock_excel_import_wizard` — Import Excel de stocks
- `reception_directe_wizard` — Réception directe fournisseur
- `retour_carton_wizard`, `retour_inventaire_wizard`, `retour_fournisseur_wizard` — Retours
- `reception_correction_info_wizard` — Correction de réceptions
- `rapport_retours_receptions_wizard` — Rapport retours/réceptions

Workflow inventaire physique :

```

Création inventaire (draft)
  ↓ action_start
En cours (in_progress)
  ↓ Saisie des quantités physiques
action_done
  ↓ Application stock.quant._apply_inventory
Création des mouvements d'ajustement
  ↓ avco.guard vérifie : écart < ±10 M FCFA
Validation définitive

```

4.4 `custom_loyalty` — Programme de fidélité

Objectif : Système de fidélité multi-famille avec accumulation de points en caisse.

Modèles :

Modèle	Champs clés	Description
<code>loyalty.family</code>	<code>name</code> , <code>points_earned</code> , <code>price_threshold</code> , <code>product_category_ids</code>	Famille de fidélité (barème)
<code>loyalty.card</code> (hérit.)	—	Extension de la carte de fidélité
<code>pos.order</code> (hérit.)	—	Calcul des points à la commande POS
<code>pos.payment</code> (hérit.)	—	Paiement par points
<code>pos.payment.method</code> (hérit.)	—	Méthode de paiement fidélité
<code>pos.session</code> (hérit.)	—	Session fidélité
<code>res.partner</code> (hérit.)	—	Solde de points client

Logique d'attribution :

```

Montant achat → Correspondance loyalty.family (price_threshold)
  ↓
Attribution points_earned par palier
  ↓
Mise à jour loyalty.card du client

```

Données de référence : `loyalty_family_data.xml` — familles prédéfinies (ex. : 1 point / 200 FCFA).

Wizard : Import en masse de cartes de fidélité via Excel.

4.5 `custom_food_credit` — Crédit alimentaire

Objectif : Gestion des avantages en nature (tickets restaurant / crédit alimentaire) des employés.

Modèles :

Modèle	Champs clés	Description
<code>food.credit</code>	<code>name</code> , <code>partner_company_id</code> , <code>start</code> , <code>end</code> , <code>amount</code> , <code>state</code> , <code>line_ids</code>	Enveloppe mensuelle par société
<code>food.credit.line</code>	<code>partner_id</code> , <code>amount</code> , <code>solde</code> , <code>amount_used</code> , <code>move_ids</code>	Ligne par employé
<code>limit.credit</code>	—	Plafond de crédit par employé
<code>account.journal</code> (hérit.)	—	Journal dédié crédit alimentaire
<code>account.payment</code> (hérit.)	—	Paielement crédit alimentaire
<code>pos.payment</code> (hérit.)	—	Paielement POS par crédit alimentaire

Workflow mensuel :

```
Wizard generate_credit_food
    ↓ Pour chaque société avec amount_food > 0
    Création food.credit (draft)
    ↓ Génération food.credit.line par employé (partenaire enfant)
    État in_progress
    ↓ Consommation en caisse POS ou facturation
    Suivi : solde = amount - amount_used
    ↓ Wizard generate_invoice_credit_food
    Génération des factures comptables
```

Assets POS : `paymentScreenCreditFood.js`, `partner_line_food_credit.xml`,
`receipt_food_credit.xml`

4.6 `custom_api_sage_x3` — Intégration SAGE X3

Objectif : Synchronisation bidirectionnelle entre Odoo 19 et l'ERP SAGE X3 (commandes d'achat, livraisons, comptabilité).

Modèles :

Modèle	Type	Description
<code>sage.x3.import.log</code>	Transient	Journal d'import (requêtes, réponses, erreurs)
<code>sage.x3.log.search.wizard</code>	Transient	Recherche dans les logs
<code>sage.x3.mixin</code>	Abstract	Authentification et helpers API
<code>purchase.order</code> (hérit.)	Permanent	Champs SAGE X3 sur les bons de commande
<code>account.move</code> (hérit.)	Permanent	Écritures comptables SAGE X3

Champs ajoutés à `purchase.order` :

Champ	Type	Description
<code>sage_x3_submitted</code>	Boolean	Envoyé à SAGE X3
<code>sage_x3_validated</code>	Boolean	Validé dans SAGE X3
<code>sage_x3_delivery_received</code>	Boolean	Livraison reçue depuis SAGE X3
<code>sage_x3_submitted_date</code>	Datetime	Date d'envoi
<code>sage_x3_delivery_date</code>	Datetime	Date de livraison
<code>sage_x3_response_message</code>	Text	Message retour SAGE X3
<code>sage_x3_error</code>	Text	Message d'erreur
<code>type_command</code>	Selection	Type de commande
<code>type_supplier</code>	Selection	Type fournisseur

Workflow d'intégration :

```
Bon de commande Odoo
  ↓ Validation
_submit_to_sage_x3() → POST /api/Orders/batch
  ↓ Confirmation SAGE X3
sage_x3_validated = True → button_confirm() Odoo
  ↓ CRON ou déclenchement manuel
_job_import_deliveries()
  ↓ GET /api/Orders/deliveries (filtrées par société)
_preload_data() → cache commandes + produits (anti-N+1)
  ↓ Traitement par lot de 100, commits intermédiaires
Mise à jour lignes BC (prix + qté) + stock.picking
  ↓
sage_x3_delivery_received = True
```

Authentification : Token Bearer avec TTL 1 heure, caché en mémoire via

```
_authenticate_sage_x3()
```

Endpoints utilisés :

Méthode	Endpoint	Usage
POST	<code>/api/Auth/login</code>	Obtention du token
POST	<code>/api/Orders/batch</code>	Envoi de commandes
GET	<code>/api/Orders/deliveries</code>	Récupération livraisons
POST	<code>/api/AccountingEntries/batch</code>	Écritures comptables
GET/POST	<code>/api/Customers</code>	Synchronisation clients
GET/POST	<code>/api/Items</code>	Synchronisation produits

4.7 `custom_purchase` — Achats

Objectif : Personnalisation des achats avec filtres produits, import de prix et réapprovisionnement.

Modèles héritant : `purchase.order`, `purchase.order.line`, `stock.warehouse.orderpoint`

Fonctionnalités :

- Filtrage des produits par statut X3 et statut entrepôt
- Assistant d'import des prix fournisseurs (Excel)
- Assistant de sélection du fournisseur de réapprovisionnement
- Gestion des points de commande (stock mini/maxi)

4.8 `custom_account` — Comptabilité

Objectif : Suivi budgétaire analytique et extensions comptables.

Modèles :

Modèle	Description
<code>account.move</code> (hérit.)	Extensions factures/avoirs
<code>account.budget</code> (hérit.)	Budget standard étendu
<code>budget.analytic</code>	Budget analytique par axe
<code>budget.analytic.daily</code>	Ventilation journalière du budget analytique

Règles de sécurité : Accès au budget analytique restreint par groupe.

4.9 `custom_partner` — Partenaires

Objectif : Extension du modèle partenaire pour les besoins SANGEL.

Modèles héritant : `res.partner`, `loyalty.card`, `pos.order`, `sale.order`

4.10 `custom_reports` — Rapports PDF

Objectif : Génération de rapports PDF métier via des assistants de filtrage.

Assistants disponibles (20+) :

Assistant	Rapport
<code>stock_valorise_report_wizard</code>	Valorisation du stock
<code>sale_stat_report_wizard</code>	Statistiques de ventes
<code>reception_fournisseur_wizard</code>	Réceptions fournisseurs
<code>supplier_return_report_wizard</code>	Retours fournisseurs
<code>cumul_inventory_report_wizard</code>	Cumul inventaire
<code>stock_adjustment_report_wizard</code>	Ajustements de stock
<code>stock_casse_report_wizard</code>	Casses et pertes
<code>report_daily_sales_wizard</code>	Ventes journalières
<code>cadencier_sale</code>	Cadencier des ventes
<code>catalog_product_report_wizard</code>	Catalogue produits
<code>retours_consolides_report_wizard</code>	Retours consolidés
<code>recap_caisses_wizard</code>	Récapitulatif caisses
<code>livraison_report_wizard</code>	Bons de livraison
<code>stock_product_report_wizard</code>	État des stocks produits

4.11 `custom_reliquat_report` — Rapport de reliquats

Objectif : Suivi du taux de satisfaction des commandes fournisseurs (qté commandée vs. reçue).

Modèles :

Modèle	Champs clés	Description
<code>reliquat.report</code>	<code>date_from</code> , <code>date_to</code> , <code>period_type</code> , <code>state</code> , <code>satisfaction_rate</code>	En-tête rapport
<code>reliquat.report.line</code>	<code>total_qty_ordered</code> , <code>total_qty_received</code> , <code>total_qty_pending</code>	Ligne par référence
<code>purchase.order</code> (hérit.)	—	Champ de suivi reliquat

Périodicités supportées : journalier, hebdomadaire, mensuel, trimestriel, bihebdomadaire, semestriel, annuel.

Indicateur clé : `satisfaction_rate = total_qty_received / total_qty_ordered * 100`

4.12 `custom_multi_barcode_for_products` — Multi-codes-barres

Objectif : Gérer plusieurs codes-barres par produit (référence principale + variantes + codes secondaires).

Modèles :

Modèle	Champs clés	Description
<code>product.multiple.barcodes</code>	<code>product_multi_barcode</code> (unique), <code>product_id</code> , <code>is_active_barcode</code>	Code-barres alternatif
<code>product.product</code> (hérit.)	—	Synchronisation du code-barres actif
<code>product.template</code> (hérit.)	—	Gestion des codes-barres depuis le gabarit
<code>product.label.layout</code> (hérit.)	—	Impression d'étiquettes multi-codes

Contrainte : Unicité du code-barres dans toute la base (`@api.constrains`).

Asset POS : `pos_multi_barcode.js` — reconnaissance multi-codes-barres en caisse.

Wizard : Import en masse de codes-barres via Excel.

4.13 `custom_price_change_tracker` — Suivi des prix

Objectif : Historisation de toutes les modifications de prix produits avec alertes quotidiennes.

Modèles :

Modèle	Description
<code>product.price.history</code>	Historique des prix (date, ancien prix, nouveau prix, utilisateur)
<code>product.template</code> (hérit.)	Déclenchement de l'enregistrement à chaque modification de prix

Assistants : `product_price_history_wizard`, `product_price_analysis_wizard`

Cron : Notification quotidienne des changements de prix (`ir_cron_data.xml`).

Bonus : Génération d'étiquettes codes-barres depuis l'historique.

4.14 `cfao_dashboard_report` — Tableau de bord CFAO

Objectif : Dashboard multi-société avec comparaison N/N-1 et analyse budgétaire.

Modèle : `report.cfao.dashboard`

4 périodes d'analyse : Jour / Semaine / Mois / Année

Indicateurs :

- Chiffre d'affaires (commandes de vente + commandes POS)
- Marges brutes
- Débits (engagements)
- Stocks par département
- Comparaison N-1
- Analyse budgétaire

Filtres : par département (catégorie de produit), par société

4.15 `dashboard_management_administration` — Dashboard Admin

Objectif : Tableau de bord administratif avec graphiques interactifs (Chart.js).

Modèles :

Modèle	Description
<code>managment.admin</code>	Paramètres du dashboard
<code>dashboard.data</code>	Données consolidées
<code>pos.actions.dashboard</code>	Actions POS en temps réel

Contrôleur : `ManagmentAdminController` — endpoints JSON pour les données graphiques.

Assets : Bibliothèque Chart.js, CSS/JS personnalisés pour les dashboards ventes, POS, achats.

Analyse Top 5 clients avec filtrage par date.

4.16 `fne_certification` — Certification FNE

Objectif : Certification des factures sur la plateforme FNE (administration fiscale).

Modèles :

Modèle	Description
<code>account.move</code> (hérit.)	Champs FNE (numéro certification, statut)
<code>account.tax</code> (hérit.)	Configuration TVA FNE
<code>fne.config.setting</code>	Paramètres de connexion FNE
<code>res.partner</code> (hérit.)	Données de certification partenaire

Assistants : `fne_certification_wizard` (certification), `fne_certification_refund` (avoirs)

Rapport : `report_invoice_fne` — facture au format FNE.

4.17 `stock_avco_correction` — Correction AVCO

Objectif : Correction des coûts moyens pondérés (AVCO) suite à la migration depuis ProgMag.

Fonctionnalités :

- Import Excel de corrections AVCO
- Règle de tolérance à 5 % (prix conservé si écart acceptable)
- Sauvegarde du prix d'origine pour la traçabilité
- Isolation multi-société

Assistants :

- `stock_avco_import_wizard` — Import des corrections
- `sale_margin_recompute_wizard` — Recalcul des marges
- `product_status_import_wizard` — Import du statut produits

4.18 `pos_history_import` — Import historique POS

Objectif : Injection en masse d'un historique de ventes POS depuis Excel (migration de données).

Modèles :

Modèle	Description
<code>pos.import.wizard</code>	Assistant principal d'import
<code>pos.import.preview.line</code>	Lignes de prévisualisation avant import

Processus :

1. Téléchargement du gabarit Excel
2. Remplissage (référence produit, client, montant, paiement, date)
3. Chargement et prévisualisation
4. Import → création automatique de `pos.session`, `pos.order`, `pos.order.line`, `pos.payment`
5. Journal d'erreurs détaillé

Regroupement : par jour ou par mois.

Dépendance Python : `openpyxl`

4.19 `hide_menu_user` — Masquage de menus

Objectif : Contrôle de la visibilité des menus par utilisateur (indépendant des groupes).

Modèles héritant : `res.users`, `ir.ui.menu`

4.20 `queue_job` — File de travaux asynchrones

Objectif : Exécution asynchrone de traitements lourds (module OCA communautaire).

Modèles :

Modèle	Description
<code>queue.job</code>	Travaux en attente / en cours / terminés
<code>queue.job.channel</code>	Canaux de priorité
<code>queue.job.function</code>	Registre des fonctions de travaux
<code>queue.job.lock</code>	Verrous anti-doublon

Utilisé par : `custom_api_sage_x3` pour l'import asynchrone des livraisons SAGE X3.

5. Modèles de données clés

Schéma `food.credit`

```
food.credit
├─ name          : Char (auto-séquence)
├─ partner_company_id : Many2one(res.partner)
├─ start / end    : Datetime
├─ amount        : Float (enveloppe mensuelle)
├─ total_amount_limit : Float (computed)
├─ amount_used    : Float (computed, somme lignes)
├─ invoiced      : Boolean
├─ move_id       : Many2one(account.move)
├─ state         : Selection [draft, in_progress, close, done]
└─ line_ids      : One2many(food.credit.line)

food.credit.line
├─ partner_id    : Many2one(res.partner) ← employé
├─ amount        : Float
├─ start / end    : Datetime
├─ food_id       : Many2one(food.credit)
├─ solde         : Float (computed = amount - amount_used)
├─ amount_used   : Float
└─ move_ids      : Many2many(account.move)
```

Schéma `sale.promotion`

```
sale.promotion
├─ name          : Char (code promo)
├─ company_ids   : Many2many(res.company)
├─ date_start / date_end : Date
├─ line_ids      : One2many(sale.promotion.line)
├─ apply_in_pos  : Boolean
└─ active        : Boolean

sale.promotion.line
├─ promotion_id  : Many2one(sale.promotion)
├─ product_id    : Many2one(product.template)
├─ discount      : Float [0-100]
├─ promo_ht      : Float
├─ promo_ttc     : Float (inverse)
├─ promo_pa      : Float (coût promo)
├─ price_ht / price_ttc : Float (computed depuis produit)
├─ coeff         : Float (coefficient)
├─ promo_coeff   : Float
├─ promo_tx_marque : Float (taux de marque)
├─ qty_available : Float (computed, stock réel)
└─ virtual_available : Float (computed, stock prévisionnel)
```

Schéma `physical.inventory`

```
physical.inventory
├─ name : Char
├─ code_inventory_id : Many2many(code.inventory)
├─ code_category_id : Many2one(code.category.inventory)
├─ team_inventory_id : Many2one(team.inventory)
├─ inventory_mode : Selection [normal, libre]
├─ state : Selection [draft, in_progress, done]
├─ line_quant_ids : One2many(stock.quant)
├─ physical_line_ids : One2many(physical.inventory.line)
├─ company_id : Many2one(res.company)
├─ date : Datetime
├─ date_done : Datetime
├─ is_negative_stock : Boolean
└─ note : Text
```

Schéma `purchase.order` (champs SAGE X3)

```
purchase.order (hérit.)
├─ sage_x3_submitted : Boolean
├─ sage_x3_validated : Boolean
├─ sage_x3_delivery_received : Boolean
├─ sage_x3_submitted_date : Datetime
├─ sage_x3_delivery_date : Datetime
├─ sage_x3_response_message : Text
├─ sage_x3_error : Text
├─ type_command : Selection
└─ type_supplier : Selection
```

6. Workflows métier

6.1 Commande d'achat → SAGE X3 → Livraison

1. Création du bon de commande Odoo
2. Validation → `_submit_to_sage_x3()` → POST `/api/Orders/batch`
3. Réception de la confirmation SAGE X3
 - `sage_x3_validated = True`
 - `button_confirm()` → BC confirmé dans Odoo
4. CRON ou déclenchement manuel : `_job_import_deliveries()`
 - GET `/api/Orders/deliveries` (par société, depuis `last_import_date`)
 - `_preload_data()` (cache anti-N+1)
 - Traitement par lots de 100 avec commits intermédiaires
 - Mise à jour lignes BC (prix + qté) + création `stock.picking`
 - `sage_x3_delivery_received = True`

6.2 Session POS et clôture

1. Ouverture de session POS (caissière/superviseur)
2. Vente : scan multi-codes-barres → recherche produit
 - Application auto promo (si `apply_in_pos = True`)
 - Application auto taxe AIRSI (selon client + produit)
3. Paiement : espèces / carte / crédit alimentaire / fidélité
 - Conversion multi-devises si paiement en devise étrangère
4. Clôture :
 - `rapport_validation` (rapport des validations)
 - `cloture_caisse` (rapport de caisse)
 - Réconciliation des paiements

6.3 Génération du crédit alimentaire mensuel

1. Wizard `generate_credit_food`
2. Pour chaque société avec `amount_food > 0` :
 - Création `food.credit` (draft)
 - Génération `food.credit.line` par employé (partenaire enfant)
3. Passage en état `in_progress`
4. Consommation en caisse POS → déduction du solde
5. Wizard `generate_invoice_credit_food`
 - Génération des factures comptables par enveloppe
6. État done après facturation complète

6.4 Inventaire physique

1. Création de l'inventaire (draft)
 - Affectation : équipe, codes, catégorie
 2. `action_start` → `in_progress`
 3. Saisie des quantités physiques par ligne
 4. `action_done` :
 - `_apply_inventory()` sur `stock.quant`
 - Création des mouvements d'ajustement
 - `avco.guard` : vérification écart $AVCO < \pm 10 \text{ M FCFA}$
 5. Validation définitive
-

7. Sécurité et droits d'accès

Groupes personnalisés (custom_pos)

Groupe	Lecture	Écriture	Suppression	Périmètre
caissiere	POS orders, sessions	POS orders limité	—	Caisse uniquement
superviseur	POS, stock, comptes	POS, stock, comptes	POS, stock	Supervision point de vente
assistant_magasin	Ventes, achats, stock, POS	Ventes, achats, stock, POS	Ventes, achats, stock	Entrepôt et vente
responsable_magasin	Tout	Tout	Tout	Gestion complète magasin
commercial	Ventes, comptes (limité)	Ventes, comptes (limité)	Ventes	Commercial terrain

Règles de données (Record Rules)

- Stocks et inventaires : isolation par société (`company_id`)
- Transferts inter-sociétés : restriction aux sociétés autorisées
- Sécurité produits : visibilité selon le statut produit

8. APIs et intégrations externes

SAGE X3 REST API

URL de base : `http://172.16.2.150:8030` (configurable via paramètre système)

Authentification : Bearer token (TTL 1h, caché en mémoire)

Format des commandes :

```
{
  "commandes": [{
    "siteVente": "VRIDI",
    "DateCommande": "2026-05-22T10:00:00",
    "Client": "YOP01",
    "Devise": "XOF",
    "Magasin": "PRINCIPAL",
    "ReferenceCommandeClient": "PO123456",
    "items": [{
      "ligne": 1000,
      "article": "PROD001",
      "TexteLigne": "Libellé produit",
      "quantite": 10
    }]
  }]
}
```

FNE (Certification fiscale)

- Workflow de certification des factures et avoirs
- Configuration de la taxe TVA FNE
- Numéro de certification stocké sur `account.move`

POS Controllers internes

Route	Méthode	Description
<code>/pos_promo/check_3x4</code>	POST	Vérifie la règle promotionnelle 3×4
<code>/pos/active_currencies</code>	GET	Retourne devises actives + taux
<code>/pos/convert_currency</code>	POST	Convertit un montant en devise locale

9. Personnalisations du POS (JavaScript)

Patches JavaScript (OWL / Odoo 17+)

Fichier	Rôle
<code>payment_screen_patch.js</code>	Personnalisation de l'écran de paiement
<code>currency_payment_screen_patch.js</code>	Popup de conversion multi-devises
<code>closing_popup_patch.js</code>	Popup de clôture de session
<code>money_details_popup_patch.js</code>	Détail du fond de caisse
<code>airsi_patch.js</code>	Application automatique de la taxe AIRSI
<code>promotion_pos_patch.js</code>	Application des promotions de vente
<code>OrderlineCustom.js</code>	Personnalisation des lignes de commande
<code>ProductScreen.js</code>	Écran de sélection produit
<code>loyalty_single_line.js</code>	Affichage des points fidélité
<code>pos_navbar_patch.js</code>	Barre de navigation (restrictions par rôle)
<code>cashbox_button.js</code>	Ouverture manuelle du tiroir (Alt+C)
<code>logout_button.js</code>	Bouton de déconnexion utilisateur
<code>price_ttc_patch.js</code>	Affichage du prix TTC
<code>ticket_screen_refund_patch.js</code>	Gestion des remboursements
<code>pos_multi_barcode.js</code>	Reconnaissance multi-codes-barres
<code>paymentScreenCreditFood.js</code>	Paiement par crédit alimentaire
<code>global_discount_patch.js</code>	Remise globale POS

Templates XML POS

Fichier	Rôle
<code>currency_conversion_popup.xml</code>	Popup de conversion de devise
<code>cash_move_hide_cash_in.xml</code>	Masquage de l'entrée de caisse
<code>custom_receipt_header.xml</code>	En-tête personnalisé des tickets
<code>orderline_customization.xml</code>	Ligne de commande enrichie
<code>pos_receipt_custom.xml</code>	Ticket de caisse personnalisé
<code>pos_navbar_caissiere_patch.xml</code>	Navbar restreinte caissière
<code>partner_line_food_credit.xml</code>	Ligne client avec crédit alimentaire
<code>receipt_food_credit.xml</code>	Ticket avec crédit alimentaire

10. Rapports et tableaux de bord

Rapports PDF (Qweb)

Catégorie	Rapports
Stock	Valorisation, ajustements, casses, état produits
Ventes	Journalier, statistiques, cadencier
Achats	Réceptions fournisseurs, retours fournisseurs
Inventaire	Inventaire physique, décompte, bon réception
Caisse	Récapitulatif caisses, retours/réceptions
Reliquat	Taux de satisfaction commandes
Catalogue	Export catalogue produits
FNE	Facture au format certification FNE
Crédit alimentaire	Plafonds, factures crédit

Tableaux de bord

Dashboard	Indicateurs
CFAO Dashboard	CA, marges, débits, stocks par société/département (J/S/M/A + N-1)
Management Admin	KPIs ventes, achats, POS avec graphiques Chart.js, Top 5 clients
POS Actions	Actions POS en temps réel, métriques session

11. Tâches planifiées (Crons)

Cron	Module	Fréquence	Action
AVCO Guard	<code>custom_stock</code>	Quotidien	Vérifie les écarts AVCO > ±10 M FCFA
Price Change Notif.	<code>custom_price_change_tracker</code>	Quotidien	Notification des modifications de prix
Reliquat Report	<code>custom_reliquat_report</code>	Configurable	Génération automatique des rapports
SAGE X3 Import	<code>custom_api_sage_x3</code>	Planifié	Import des livraisons depuis SAGE X3
Food Credit Generate	<code>custom_food_credit</code>	Mensuel	Génération des crédits alimentaires

12. Dépendances externes Python

Bibliothèque	Usage
<code>openpyxl</code>	Lecture/écriture Excel (imports, exports)
<code>requests</code>	Appels HTTP vers SAGE X3
<code>python-dateutil</code>	Calculs de dates (relativedelta pour mensuel)

Installation :

```
pip install openpyxl requests python-dateutil
```

13. Paramètres système

Les paramètres suivants doivent être configurés via **Paramètres** → **Technique** → **Paramètres système** :

Clé	Valeur par défaut	Description
<code>sage_x3.base_url</code>	<code>http://172.16.2.150:8030</code>	URL de base de l'API SAGE X3
<code>sage_x3.username</code>	<code>odoo</code>	Identifiant SAGE X3
<code>sage_x3.password</code>	<code>InterfaceX3_Odoo</code>	Mot de passe SAGE X3
<code>sage_x3.last_import_date.company_{id}</code>	ISO timestamp	Date du dernier import par société

Important : Le mot de passe SAGE X3 est stocké en clair dans les paramètres système. Il est recommandé de le stocker dans un coffre-fort de secrets en production.